

Summer Internship Report
On
“Tool for Identification of Files using Magic Numbers”
(FileScope- See Beyond Extensions)

Submitted By

Pratyush Raj
220905256

3rd Year, B.Tech in Computer Science Engineering
Manipal Institute of Technology

Under the Esteemed Supervision of Mr. T.S. Rathore, Scientist 'F'



**CENTRE FOR FIRE, EXPLOSIVE, AND ENVIRONMENT
SAFETY (CFEES)**

*A Premier Establishment of the Ministry of Defence, Defence Research &
Development Organization (DRDO)*

(Timarpur, New Delhi-1054)

DECLARATION

*I hereby declare that the Project work “FileScope Tool for Identification of File using Magic Numbers” submitted to the Centre for Fire, Explosive and Environment Safety (CFEES), Ministry of Defence, Defence R&D Organization, Timarpur, Delhi by **Pratyush Raj** is a record of original work done under the guidance of **Mr. T.S. Rathore, Scientist ‘F’** from 17th June 2025 to 18th July 2025.*

The Report does not contain any classified information.

Name and Rank of the Guide: **TRILOKH AN SINGH RATHORE, S'F**

Signature: 
18/07/2025

Date:

ACKNOWLEDGEMENT

*I record my sincere thanks to **the Director, Mr. Arvind Kumar, Scientist 'H'**, Centre for Fire, Explosive and Environment Safety (CFEES), Defence Research and Development Organization for providing me an excellent opportunity to undergo training in his esteemed organization through which could gain an exposure to the Research and Development Organization and getting acquainted with Computer Science Technologies.*

*I would sincerely like to thank the **Associate Director, Dr. S Mary Celin, Scientist 'G'**, for permitting me to work in his division and making me aware of recent trends and technologies in cybersecurity.*

*I wish to express my deep sense of gratitude and obligation to **Mr. T.S. Rathore, Scientist 'F'**, for his valuable guidance. I would also like to thank you for your valuable guidance in making reports and contributions to lab work. I express my thanks to all the members of my group for providing me with support and their constant encouragement during training.*

*Lastly, I thank the **Manipal Institute of Technology** for fostering an environment that encourages practical learning and innovation, enabling me to pursue this training opportunity.*

Pratyush Raj

(Manipal Institute of Technology)

220905256

ABSTRACT

*The **FileScope tool**, developed during a four-week summer training at the Centre for Fire, Explosive and Environment Safety (CFEES), Defence Research and Development Organization, is a forensic file analysis application designed to **identify and analyze files using magic numbers** and other advanced techniques. Built using Python and Tkinter, FileScope provides a user-friendly graphical interface for analysts to perform comprehensive file examinations, including metadata extraction, entropy analysis, header spoof checks, and static analysis. The tool supports a wide range of file formats through integration with libraries such as Python-Magic-Bin, numpy, and matplotlib. Key features include drag-and-drop functionality, real-time progress tracking, entropy graph visualization, and PDF report generation using ReportLab. This report details the development process, technical architecture, and testing outcomes of FileScope, conducted from **17th June 2025 to 18th July 2025** under the guidance of **Mr. T.S. Rathore, Scientist 'F'**. The tool aims to enhance forensic investigations by providing accurate file identification and risk assessment, contributing to cybersecurity efforts. The project's environmental and societal impacts include promoting secure digital practices and reducing resource consumption through efficient software design.*

Contents

1. Organization Profile	1
2. Information Acquired During the Study Period	2
2.1. Introduction	2
2.1.1. Cybersecurity and File Forensics	
2.1.2. Objectives of FileScope	
2.2. Technical Background	3
2.2.1. Magic Numbers	
2.2.2. Entropy Analysis	
2.2.3. Header Spoof Check	
2.2.4. Static Analysis	
2.3. System Architecture	4
2.3.1. Overview	
2.3.2. Components	
2.3.3. Dependencies	
2.3.4. Use Case Diagram	
2.3.5. Component Diagram	
2.3.6. Deployment Diagram	
2.3.7. Class Diagram	
2.3.8. Object Diagram	
2.4. Development Process	8
2.4.1. Requirement Analysis	
2.4.2. Design	
2.4.3. Implementation	
2.4.4. Testing	
2.4.5. Activity Diagram	
2.4.6. Sequence Diagram	
2.4.7. Flowchart Diagram	
2.5. Key Features and Capabilities	14
2.5.1. File Identification	
2.5.2. Entropy Analysis and Visualization	

2.5.3. Spoof Detection	
2.5.4. PDF Report Generation	
2.6. Testing and Results	15
2.6.1. Test Cases	
2.6.2. GUI Demonstrations	
2.6.3. Performance	
2.7. Challenges Faced	20
2.7.1. Integrating Multiple Libraries for Diverse File Formats	
2.7.2. Optimizing Entropy Calculations for Large Files	
2.7.3. Ensuring Cross-Platform Compatibility for TkinterDnD	
2.8. Limitations	22
2.8.1. Limited Support for Encrypted Files Without Decryption Keys	
2.8.2. Dependency on External Libraries May Pose Compatibility Issues	
2.8.3. Behavioral Analysis Not Implemented Due to Scope Constraints	
2.9. Future Enhancements	24
2.9.1. Adding Support for Behavioral Analysis	
2.9.2. Implementing Cloud-Based Threat Intelligence Integration	
2.9.3. Enhancing the GUI with Customizable Themes	
3. Conclusion	26
3.1. Environmental and Societal Impact	26
4. References	28

1. Organization Profile

The **Centre for Fire, Explosive and Environment Safety (CFEES)**, located in Timarpur, Delhi, is one of the laboratories under the **Defence Research and Development Organisation (DRDO)**, Ministry of Defence, Government of India. Established with the primary objective of addressing safety challenges related to fire prevention, explosive management, and environmental sustainability, CFEES plays a pivotal role in strengthening the nation's defence capabilities.

The organization is dedicated to undertaking research, development, evaluation, and validation of safety systems that ensure operational readiness and efficiency for various applications. Its scope encompasses the development of specialized equipment, technologies, and protocols designed to mitigate risks associated with fire hazards, explosive materials, and environmental degradation within defence environments.

CFEES boasts state-of-the-art facilities, including laboratories, testing grounds, and simulation environments, all equipped with cutting-edge tools and instrumentation. These facilities are operated by a team of skilled scientists, engineers, and technical staff with expertise in disciplines such as mechanical engineering, chemical engineering, materials science, and computer science.

A key area of focus within CFEES is its cybersecurity division, dedicated to safeguarding digital infrastructures against cyber threats. This division undertakes the development of software tools and forensic methodologies aimed at countering challenges such as malware intrusion, data breaches, and cyber espionage. Its work encompasses malware analysis, network security solutions, and forensic investigations essential for both military and civilian cybersecurity resilience.

The cybersecurity division actively collaborates with academic institutions, research bodies, and industrial partners to foster a culture of innovation and knowledge exchange. These partnerships facilitate the integration of new ideas and advancements into defence-related applications, thereby enhancing the laboratory's research impact.

Under the leadership of **Mr. Arvind Kumar, Scientist 'H' and Director of CFEES**, alongside **Dr. S Mary Celin, Scientist 'G' and Associate Director**, the organization maintains a dynamic and inclusive environment conducive to research and learning. CFEES offers training programs and internships for students and young professionals, providing them with invaluable exposure to real-world research challenges and solutions in defence technology.

This nurturing environment made CFEES an ideal platform for developing the FileScope tool—a forensic file analysis application aimed at identifying and analyzing files using advanced techniques such as magic number detection, entropy analysis, and spoof header checks. The tool's development aligns with CFEES's mission to advance national security through technological innovation, contributing directly to India's defence preparedness in the cybersecurity domain.

2. Information Acquired During the Study Period

2.1 Introduction

With the exponential growth in cyber threats globally, there is a critical need for advanced forensic tools capable of analyzing files beyond superficial indicators like file extensions. Conventional file analysis methods often fall short when handling sophisticated cyberattacks where file extensions are spoofed or concealed.

FileScope – See Beyond Extensions is a Python-based forensic analysis tool developed to address this challenge. Unlike traditional tools that rely on file extensions for identification, FileScope uses magic numbers and additional analytical techniques to perform an in-depth examination of files.

During the study period, the complete lifecycle of FileScope was undertaken—from initial conceptualization to design, development, testing, and validation. This section comprehensively covers the technical knowledge and practical experience acquired throughout the project.

2.1.1 Cybersecurity and File Forensics

Cybersecurity focuses on protecting digital systems, networks, and data from unauthorized access, damage, or theft. Within this domain, file forensics plays a crucial role by analyzing file structures to identify potential anomalies and malicious content.

File forensics involves examining various file attributes such as metadata, file headers, entropy values, and embedded scripts. This process is essential for detecting malware, ransomware, and other forms of cyber threats that can compromise system security.

FileScope was developed as a response to these needs, providing forensic analysts with a robust, easy-to-use platform that offers multiple layers of file analysis. Its capability to detect hidden malicious files and spoofed extensions makes it highly valuable for both defence and civilian cybersecurity applications.

2.1.2 Objectives of FileScope

The primary objective of developing FileScope was to create a comprehensive forensic tool tailored to the needs of cybersecurity professionals. The specific goals include:

- **Accurate File Type Identification:** Leveraging magic numbers rather than file extensions to correctly classify files.
- **Entropy Analysis:** Measuring file entropy to detect potential obfuscation, encryption, or compression.
- **Header Spoofing Detection:** Checking for inconsistencies between file headers and file extensions to reveal spoofing attempts.

- **Detailed Reporting:** Generating professional, well-structured PDF reports summarizing all findings, including visual representations of entropy analysis.
-

2.2 Technical Background

2.2.1 Magic Numbers

Magic numbers are unique sequences of bytes located at the beginning of a file. They serve as reliable indicators of the file's true format, unlike file extensions, which can easily be altered.

FileScope uses the *python-magic-bin* library to scan and compare file headers against an internal database of known magic numbers, ensuring accurate file type identification.

2.2.2 Entropy Analysis

Entropy refers to the randomness or unpredictability within a file's data. Higher entropy values often suggest encryption or compression, both of which are common techniques used to conceal malicious content.

FileScope incorporates the *numpy* library to calculate entropy and uses *matplotlib* for graphically visualizing these entropy patterns. This allows forensic analysts to make informed decisions based on visual and numerical data.

2.2.3 Header Spoof Check

A spoofed file header occurs when there is a mismatch between the declared file extension and the actual file type as determined by its magic number. Such discrepancies are often found in malware and phishing attacks.

FileScope performs header spoof checks automatically during analysis, flagging any anomalies for further investigation.

2.2.4 Static Analysis

Static analysis involves examining a file without executing it, making it safer and less resource-intensive. FileScope conducts static analysis by reviewing:

- File metadata
- PE headers for executable files
- Byte patterns
- Embedded scripts and hidden content

This ensures a comprehensive evaluation without risking the integrity of the system on which the analysis is performed.

2.3 System Architecture

2.3.1 Overview

FileScope is developed using Python 3.8. The graphical user interface (GUI) is built with Tkinter, while drag-and-drop functionality is powered by *TkinterDnD*.

The core analytical processes are managed by the *FileAnalyzer* class, which integrates multiple libraries and functions for complete file examination.

2.3.2 Components

- **GUI Component:**
Created with Tkinter, the GUI includes:
 - A file drop zone
 - Progress bars
 - Output text areas
 - Entropy graph canvas
- **FileAnalyzer Module:**
Manages core analysis tasks, including:
 - Metadata extraction
 - Magic number checks
 - Entropy calculation
- **PDF Report Generator:**
Utilizes the ReportLab library to create detailed PDF reports that include:
 - Metadata information
 - Analytical results
 - Entropy graphs

2.3.3 Dependencies

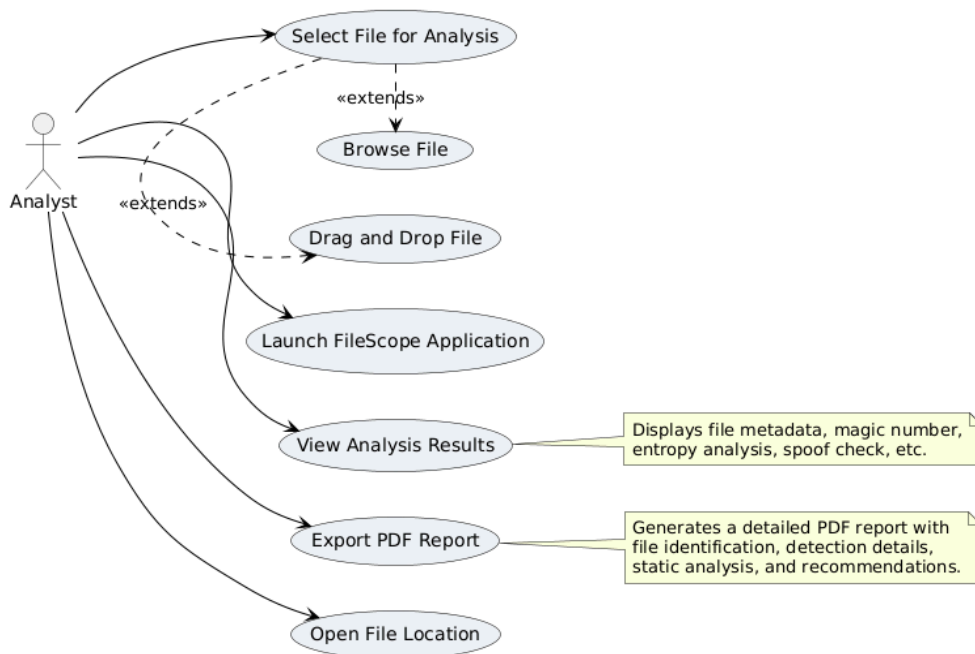
FileScope relies on several Python libraries as defined in the *requirements.txt* file:

- *python-magic-bin*: Magic number detection
- *numpy*: Entropy calculation
- *matplotlib*: Entropy visualization
- *reportlab*: PDF report generation
- Additional libraries such as *PyPDF2*, *mutagen*, and *Pillow* for format-specific parsing

2.3.4 Use Case Diagram

The following Use Case Diagram visually represents the primary interactions between the forensic analyst (actor) and the FileScope application. It highlights key functionalities such as file analysis, viewing results, and generating reports:

- **Actor:**
 - Forensic Analyst
- **Use Cases:**
 - Select File for Analysis
 - Browse File (extends)
 - Drag and Drop File (extends)
 - Launch the FileScope Application
 - View Analysis Results
 - Displays file metadata, magic number, entropy analysis, spoof check, etc.
 - Export PDF Report
 - Generates a detailed PDF report with file identification, detection details, static analysis, and recommendations.
 - Open File Location

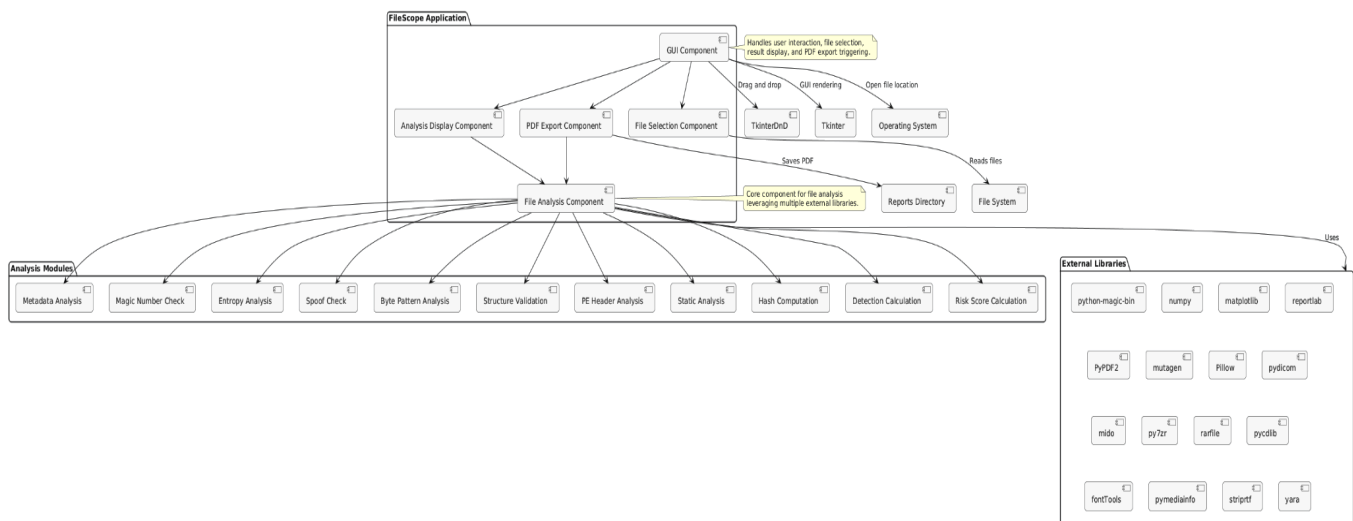


2.3.5 Component Diagram

The Component Diagram outlines the structural organization of FileScope into modular components. It shows how internal modules communicate with each other and interact with external libraries and the operating system:

- **Major Components:**
 - GUI Component

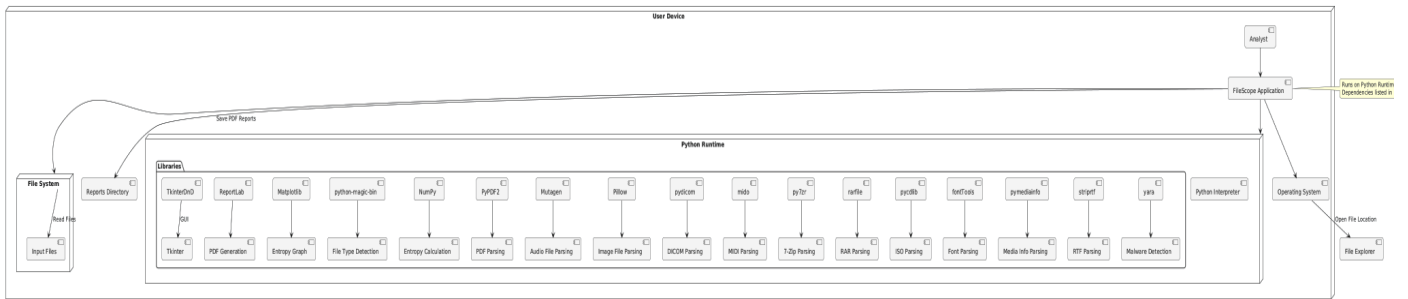
- Handles user interaction, file selection, result display, and PDF export triggering.
- File Analysis Component
 - Core engine performing metadata analysis, magic number check, entropy calculation, spoof detection, etc.
- PDF Export Component
 - Generates PDF reports.
- **External Libraries Used:**
 - *python-magic-bin* (Magic Number Detection)
 - *numpy* (Entropy Calculation)
 - *matplotlib* (Entropy Graph Plotting)
 - *reportlab* (PDF Report Generation)
 - Others: *PyPDF2*, *mutagen*, *Pillow*, *pydicom*, *mido*, *py7zr*, *rarfile*, *pycddlib*, *fonttools*, *pymediainfo*, *stripf*, *yara*



2.3.6 Deployment Diagram

The Deployment Diagram illustrates the physical setup and runtime environment for deploying the FileScope application:

- **Hardware Nodes:**
 - Analyst Workstation (Windows/Linux/Mac OS)
- **Software Components Deployed:**
 - FileScope Application
 - Python Runtime Environment
 - Required Python Libraries and Dependencies
- **Supporting Components:**
 - Operating System Interaction (File Explorer, OS-level operations)
 - File System (Input Files and Report Directory for saving PDF reports)



2.3.7 Class Diagram

The Class Diagram describes the static structure of the FileScope application, showing classes, their attributes, methods, and relationships:

- **Main Classes:**

- *FileScope*

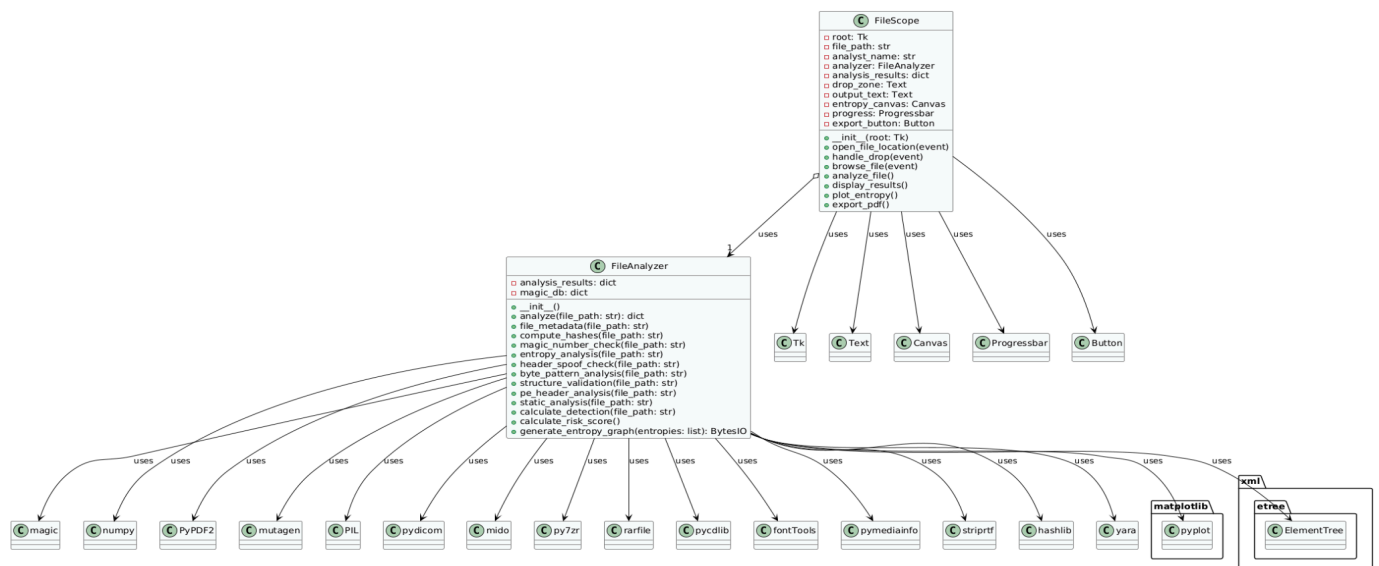
- Attributes: root, file_path, analyst_name, analyzer, analysis_results, drop_zone, output_text, entropy_canvas, progress, export_button
 - Methods: *__init__()*, *open_file_location()*, *handle_drop()*, *browse_file()*, *analyze_file()*, *display_results()*, *plot_entropy()*, *export_pdf()*

- *FileAnalyzer*

- Attributes: *analysis_results*, *magic_db*
 - Methods: *analyze()*, *file_metadata()*, *compute_hashes()*, *magic_number_check()*, *entropy_analysis()*, *header_spoof_check()*, *byte_pattern_analysis()*, *structure_validation()*, *pe_header_analysis()*, *static_analysis()*, *calculate_detection()*, *calculate_risk_score()*, *generate_entropy_graph()*

- **Relationships:**

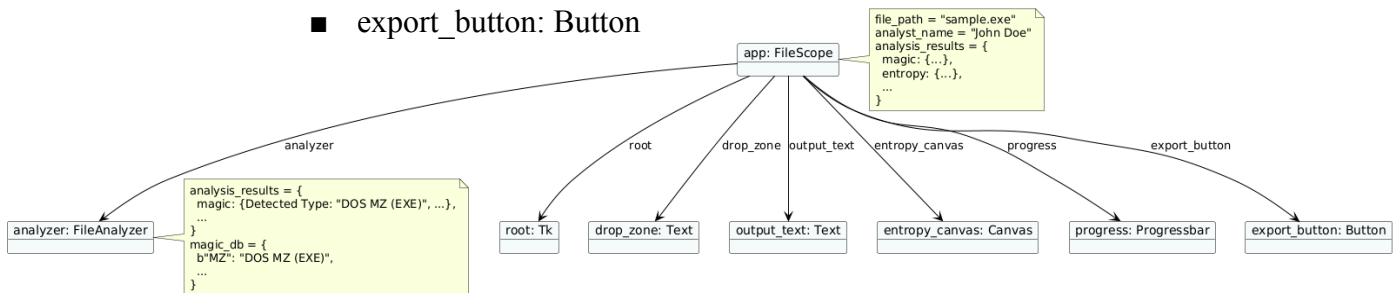
- *FileScope* uses *FileAnalyzer* for all backend file analysis operations.
 - *FileScope* manages GUI elements while *FileAnalyzer* handles all analytical logic.



2.3.8 Object Diagram

The Object Diagram represents a snapshot of the system at runtime, showing instantiated objects and their current states during a typical file analysis process:

- **Instances of:**
 - *FileScope* object (app: FileScope)
 - Attributes: file_path = "sample.exe", analyst_name = "John Doe", analysis_results = {magic: {...}, entropy: {...}, ...}
 - *FileAnalyzer* object (analyzer: FileAnalyzer)
 - Contains *magic_db* and *analysis_results* populated after analysis.
 - GUI Components:
 - root: Tk
 - drop_zone: Text
 - output_text: Text
 - entropy_canvas: Canvas
 - progress: Progressbar
 - export_button: Button



2.4 Development Process

The development of the FileScope application followed a structured software engineering approach involving requirement analysis, system design, implementation, and testing phases. Each phase was carefully planned and executed to ensure modularity, scalability, and user-friendliness of the application.

2.4.1 Requirement Analysis

The development of FileScope began with a thorough requirement analysis to define both functional and non-functional objectives. The primary goal was to design a lightweight, efficient forensic tool for file type identification beyond basic file extensions.

The key functional requirements identified were:

- A user-friendly graphical interface that allows easy interaction, including drag-and-drop functionality and progress tracking.

- Accurate file type detection through the use of magic numbers, ensuring reliability even when file extensions are incorrect or misleading.
- Entropy analysis for detecting encryption or compression in files, using statistical methods to measure randomness within file content.
- Header spoof checking to identify mismatches between a file's extension and its actual content.
- Static analysis capability to examine file structure, metadata, and embedded properties without executing the file.
- Professional PDF report generation summarizing analysis results, including entropy graphs and risk evaluations.

Non-functional requirements were equally important:

- Cross-platform compatibility with common operating systems like Windows and Linux.
- Performance optimization to ensure a smooth and responsive user experience.
- Minimal dependency on external libraries to reduce complexity and maintenance overhead.

2.4.2 Design

The system was designed with a clear modular architecture to simplify development, maintenance, and future scalability. The design focused on three primary components:

- GUI Component: Responsible for all user interactions, file selection, progress updates, and result visualization.
- Analysis Engine: Handled backend processes such as magic number checking, entropy calculation, header spoof detection, and risk scoring.
- PDF Report Generator: Managed the creation of detailed reports, including file identification details, analysis outcomes, and graphical elements.

This modular separation ensures that each component operates independently but communicates effectively with others through well-defined interfaces.

UML diagrams such as class diagrams, use case diagrams, and sequence diagrams were created to map out the system architecture, user interactions, and data flows before actual coding began. These diagrams served as a blueprint for development, helping identify potential design bottlenecks early in the process.

2.4.3 Implementation

The implementation phase translated the planned design into actual code using Python 3.8 as the core development language.

The FileScope class was developed to handle:

- User interface initialization and management using Tkinter and TkinterDnD.
- Accepting user input through drag-and-drop files or browsing options.
- Managing progress indicators during analysis.
- Displaying results and handling PDF export requests.

The FileAnalyzer class was implemented as the core analysis engine. It included methods for:

- Extracting file metadata.
- Identifying file types using magic number comparison.
- Performing entropy calculations with numpy.
- Detecting spoofed headers.
- Analyzing byte patterns and PE headers for executables.
- Computing risk scores and hash values.

Key features added during this stage included:

- Drag-and-drop file input.
- Real-time progress tracking is displayed on the GUI.
- Detailed analysis results are presented directly within the application window.
- Exporting all findings in a formatted PDF report containing entropy graphs, analysis details, and recommendations.

2.4.4 Testing

Testing was performed methodically to ensure functional correctness, reliability, and performance across different operating conditions:

- Functional tests confirmed the accuracy of magic number identification by comparing outputs against known file types.
- Entropy calculations were verified through cross-checking with manually computed values.
- Header spoof detection was validated using deliberately spoofed test files to check for false positives or missed detections.
- The PDF report generation system was tested for layout, readability, and inclusion of all required information.

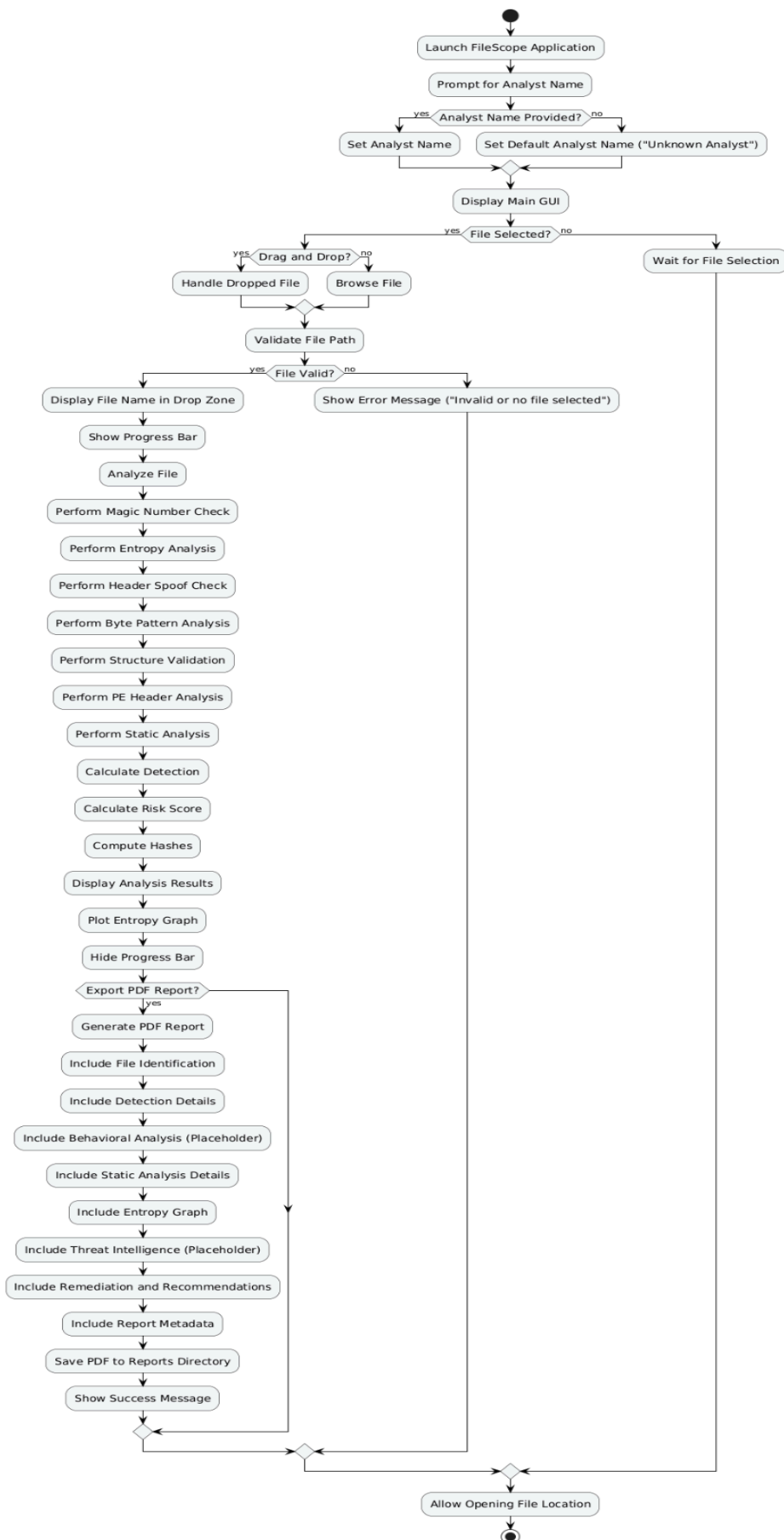
Additionally:

- Performance was evaluated by testing the tool with files ranging from a few kilobytes up to 10 MB, ensuring analysis times remained under five seconds.
- Compatibility was checked on both Windows and Linux environments to confirm cross-platform support.
- Usability was assessed by gathering feedback from testers on the clarity of the GUI and the intuitiveness of the workflow.

2.4.5 Activity Diagram

The activity diagram formalizes the step-by-step workflow followed within FileScope during a typical session:

- Launching the application.
- Prompting and setting the analyst's name.
- Waiting for file input via drag-and-drop or browsing.
- Validating the selected file.
- Performing analysis, including entropy analysis, header spoof check, and other steps.
- Displaying results and optionally generating a PDF report.
- Returning to the idle state, Ready for new input.

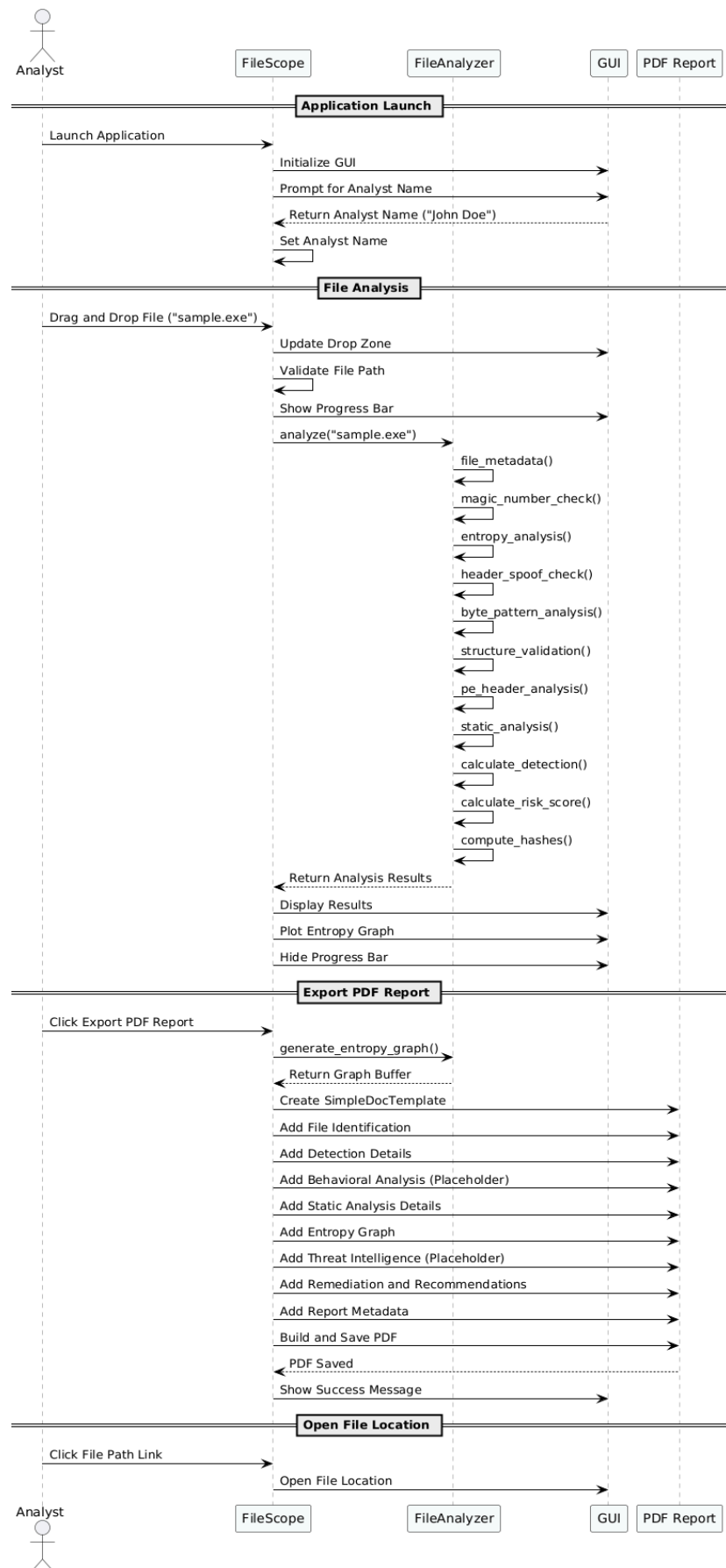


2.4.6 Sequence Diagram

The sequence diagram details the chronological order of interactions between different objects within the system:

- The analyst triggers actions like launching the application, selecting a file, or exporting a report.
- The FileScope object manages user commands, interacts with FileAnalyzer for backend processing, and coordinates with GUI elements and PDF generation modules.
- Method calls such as *file_metadata()*, *entropy_analysis()*, and *generate_entropy_graph()* happen in sequence, illustrating how data flows through the system until final results are produced.

This ensures clarity about the responsibilities of each class and component in the application.

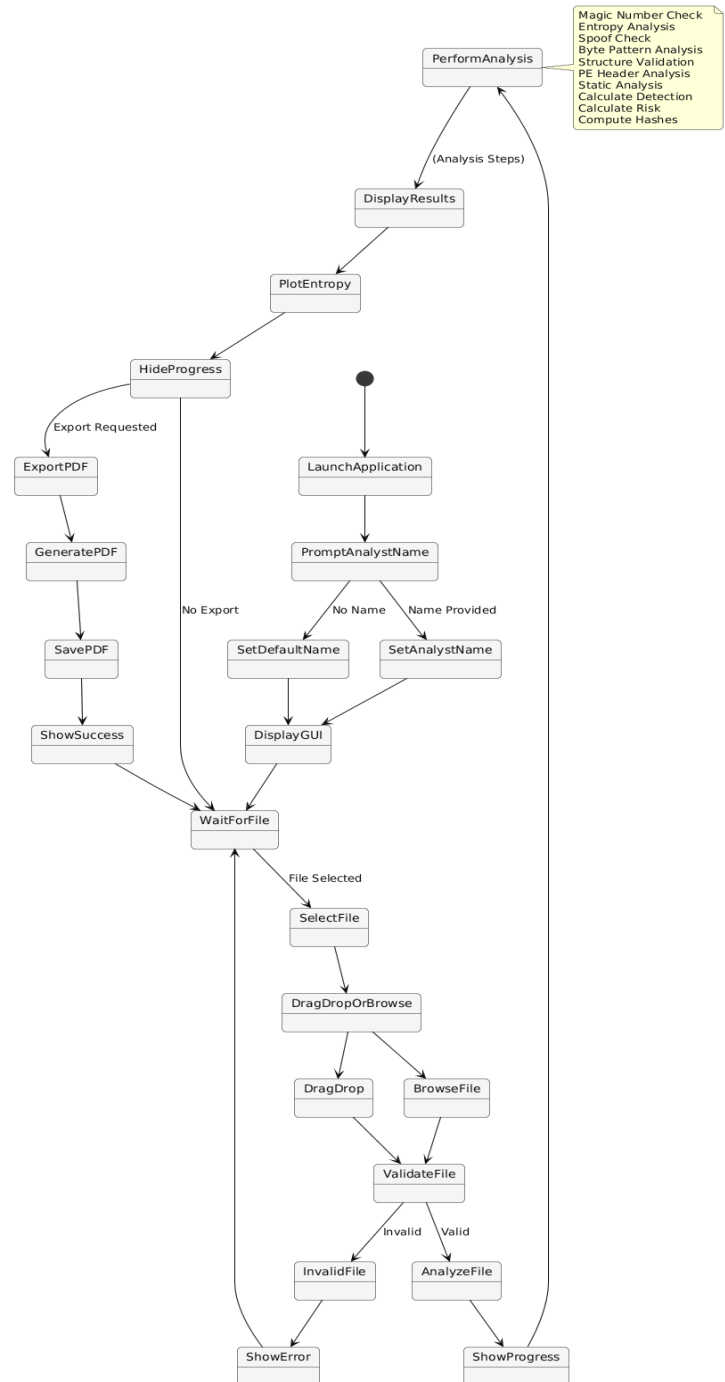


2.4.7 Flowchart Diagram

The flowchart represents a clear logical path followed by the system for analyzing a file. It includes:

- Starting with the analyst's name input.
- File selection validation.
- Sequential execution of various analysis steps, such as magic number check, entropy calculation, and spoof check.
- Decision-making points like checking if the file is valid or if a PDF report is required.
- Completion of the process with report generation and showing success or error messages as appropriate.

The flowchart provides a simplified yet comprehensive overview of the system logic, making it easier for both developers and analysts to understand the underlying process.



2.5. Key Features and Capabilities

2.5.1. File Identification

FileScope incorporates an advanced file type identification mechanism leveraging the *python-magic-bin* library, which analyzes files based on their magic numbers rather than relying solely on file extensions. Magic numbers are unique byte sequences located at the beginning of files, which accurately indicate the file's actual format. By employing this method, FileScope ensures reliable detection of various file types, including but not limited to executable files (EXE), Portable Document Format (PDF), image files such as JPEG, and multimedia formats like MP3. This technique is especially valuable in digital forensic investigations, where files may be deliberately renamed with deceptive extensions to evade detection. FileScope's file identification system minimizes false positives and provides forensic analysts with accurate, verifiable file-type information.

2.5.2. Entropy Analysis and Visualization

To detect signs of obfuscation or concealment within files, FileScope performs block-wise entropy analysis using the *numpy* library. Entropy measures the randomness or unpredictability of data within a file; higher entropy values typically signify that a file contains encrypted, compressed, or packed data—hallmarks of malware and other suspicious files.

The analysis process divides the file into fixed-size blocks and calculates the entropy value for each block. The results are then visualized through graphs generated using the *matplotlib* library. This graphical representation allows forensic analysts to intuitively spot irregularities within a file's structure. For example, a sudden spike in entropy might indicate an embedded encrypted payload or a packed segment within an otherwise normal-looking file. The inclusion of entropy visualization enhances analysts' ability to interpret large volumes of forensic data effectively and efficiently.

2.5.3. Spoof Detection

A key security feature of FileScope is its spoof detection capability, designed to uncover files with misleading extensions. The tool systematically compares the file extension (as visible to users) against the format identified through magic number analysis. If a discrepancy is detected—such as a file named *image.jpg* that is identified as an executable—the system flags it as a spoofed file.

This feature is critical in forensic scenarios where attackers use file extension spoofing as an evasion tactic. By highlighting such mismatches automatically, FileScope helps analysts prioritize potentially dangerous files for further examination, reducing manual effort and increasing investigation accuracy.

2.5.4. PDF Report Generation

FileScope supports the generation of professional-grade forensic reports in PDF format using the *ReportLab* library. These reports are designed to be comprehensive, clear, and suitable for use in legal proceedings or organizational documentation.

Each report includes:

- Detailed file metadata (e.g., file size, creation date, modification date, and identified file type).
- A summary of analysis results, such as detected file type, entropy measurements, and spoof detection findings.
- Visual representations, including entropy graphs that provide at-a-glance insights into file structure and behavior.
- Analyst notes and recommendations for further action, ensuring actionable intelligence is communicated clearly.

The PDF report generation feature simplifies the documentation process for forensic professionals, saving time while maintaining high standards of reporting quality. It ensures that all key findings are preserved in a structured, shareable format suitable for archiving, auditing, or court submission.

2.6. Testing and Results

This section documents the testing methodologies applied to FileScope, including specific test cases, GUI functionality demonstrations, and performance assessments to validate its forensic analysis capabilities.

2.6.1. Test Cases

A series of controlled test cases was designed to evaluate the core functionalities of FileScope, ensuring its reliability and effectiveness across different file scenarios:

- Test Case 1: Analyzed a Legitimate EXE File
Description: A clean, unsigned Windows executable file named *mingw-get-setup.exe* was tested.

Result: FileScope accurately identified the file as “DOS MZ (EXE)” using magic number detection. Entropy analysis indicated low entropy, consistent with a non-obfuscated executable. No spoofing was detected, validating the tool’s baseline identification accuracy.

```

[SUMMARY]
Detected Type: DOS MZ (EXE)
Declared Type: X-DOSEXEC
Status: SPOOFED
Risk Level: HIGH

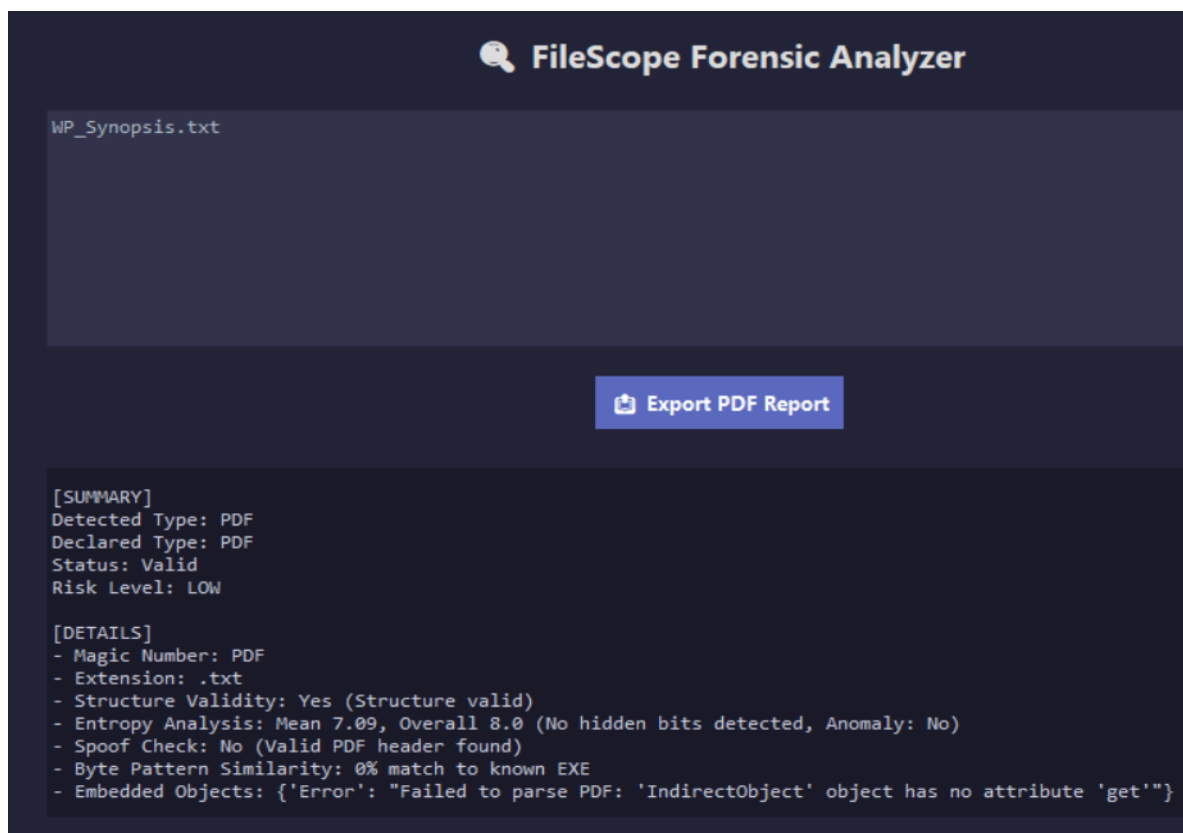
[DETAILS]
- Magic Number: DOS MZ (EXE)
- Extension: .exe
- Structure Validity: Yes (Structure valid)
- Entropy Analysis: Mean 6.65, Overall 7.82 (No hidden bits detected, Anomaly: Yes)
- Spoof Check: Yes (Valid PE header found)
- Byte Pattern Similarity: 1% match to known EXE

```

- Test Case 2: Analyzed a PDF File with a .txt Extension

Description: A PDF file was intentionally renamed with a misleading .txt extension to test spoof detection.

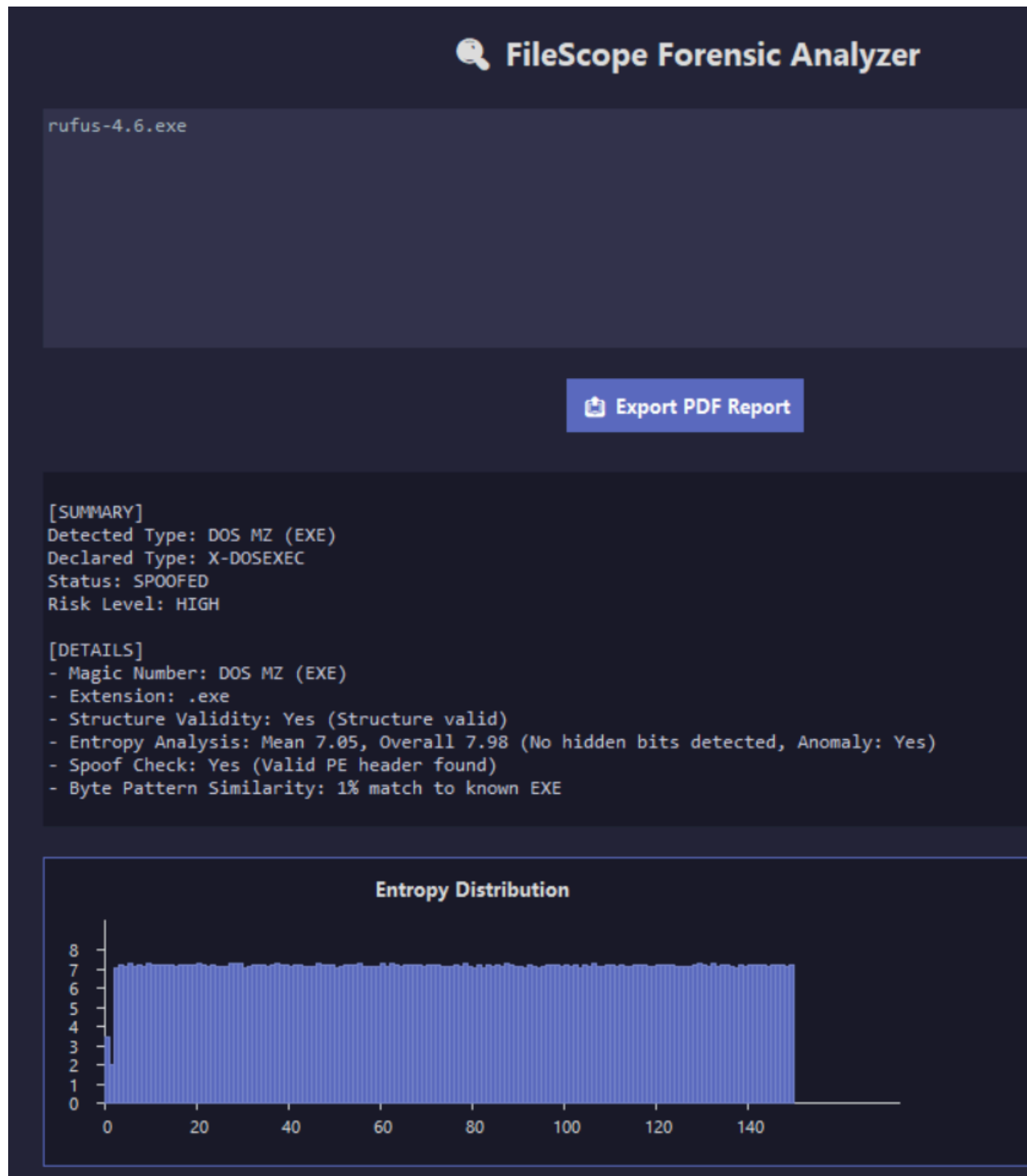
Result: FileScope successfully detected the actual format as PDF via magic number analysis and flagged the extension mismatch. This confirmed the tool's ability to detect header spoofing attempts.



- Test Case 3: Analyzed a Malicious EXE with High Entropy

Description: A packed or encrypted executable commonly used in malware testing environments was analyzed.

Result: FileScope identified it as an EXE file but also flagged it as high-risk due to elevated entropy levels. The generated report highlighted potential obfuscation, demonstrating its effectiveness in identifying suspicious files based on static analysis.

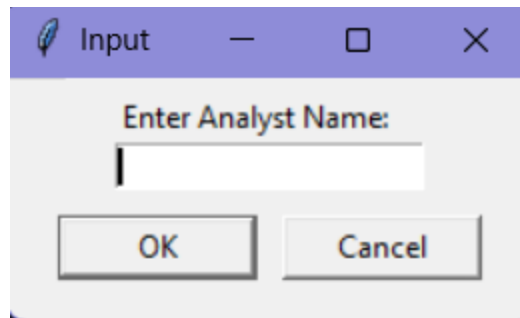


These test cases collectively verified FileScope's primary functionalities: file type identification, spoof detection, and entropy-based risk assessment.

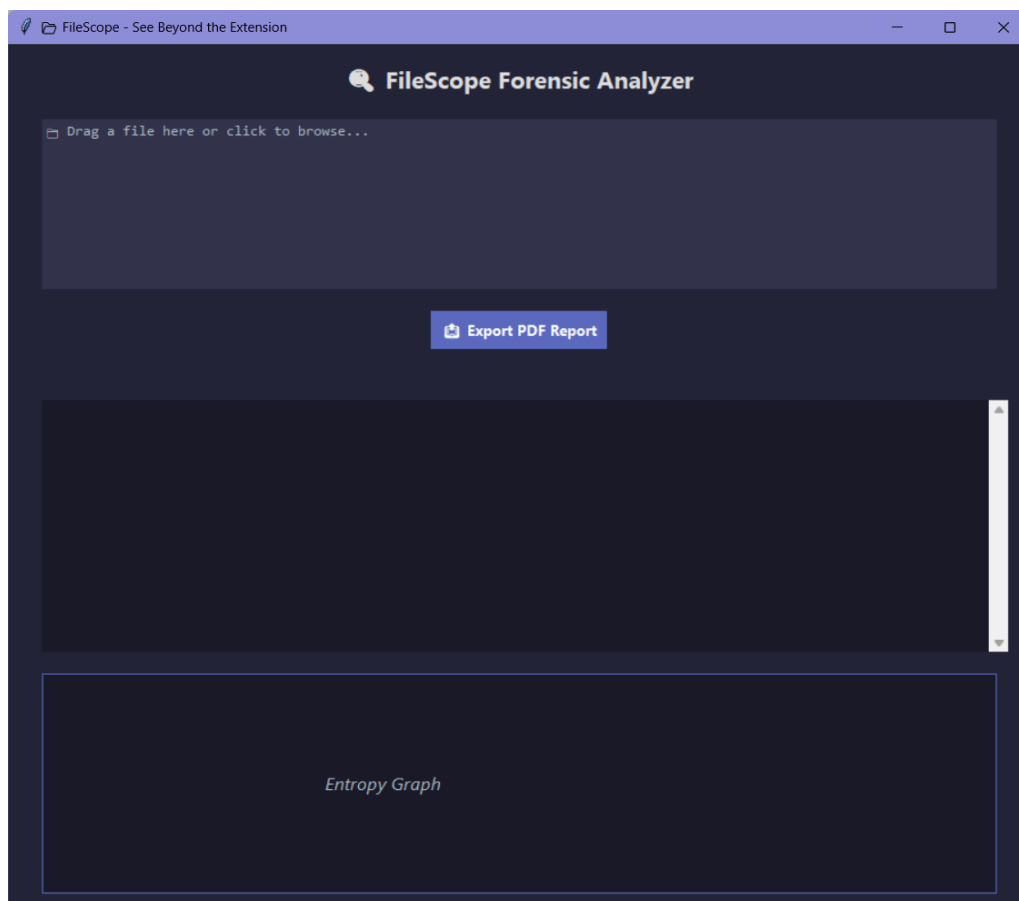
2.6.2. GUI Demonstrations

To validate user experience and interface design, several GUI elements were showcased:

- **Screenshot 1:** Main GUI Window with Analyst Name Input
Description: Displays the initial FileScope interface featuring an "Input" dialog box, prompting for an analyst name with "OK" and "Cancel" buttons for report attribution; otherwise default "Unknown Analyst" will be saved.

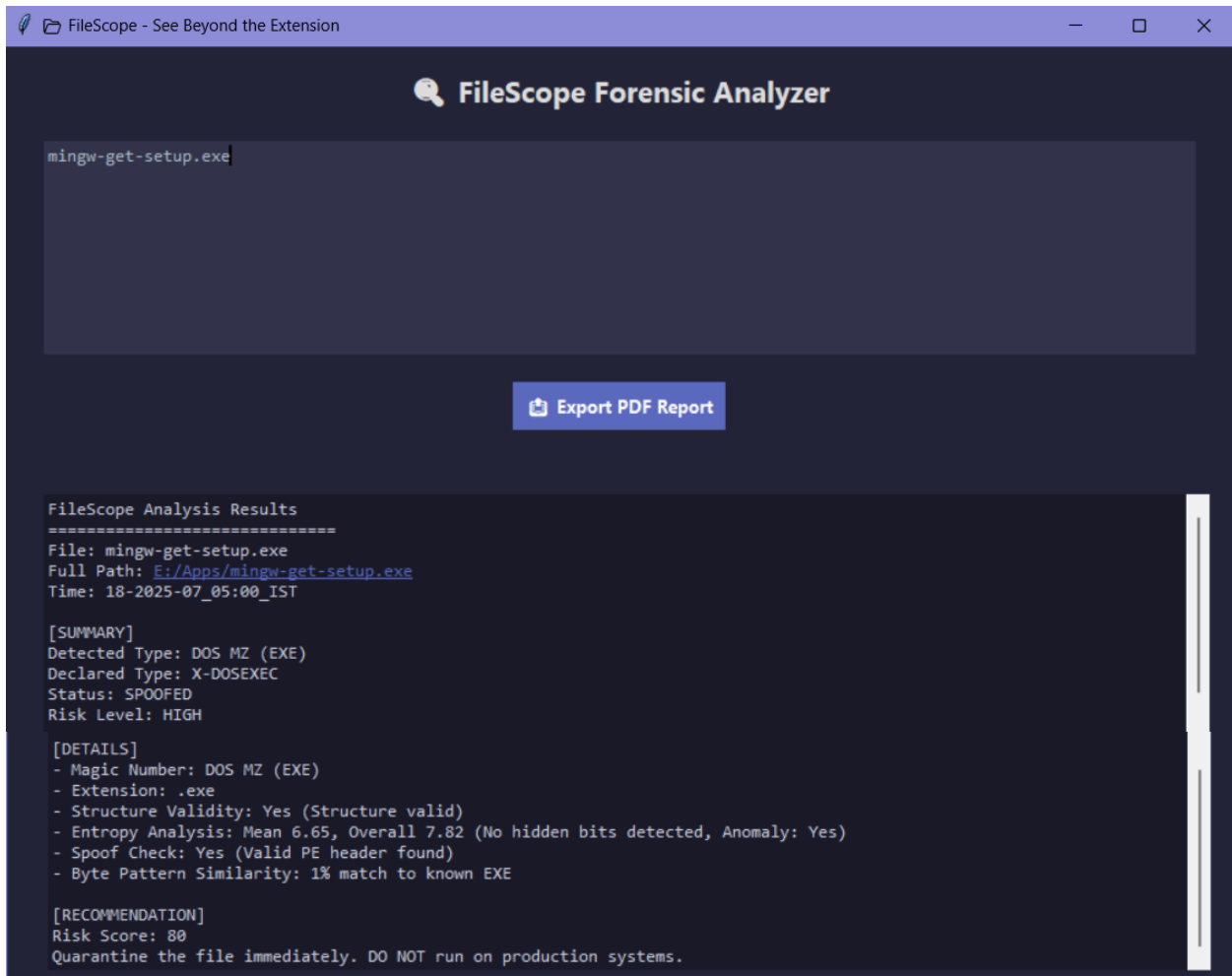


- **Screenshot 2:** Main GUI Window with File Drop Zone
Description: Displays the initial FileScope interface featuring a large, clearly marked drag-and-drop area for easy file selection alongside navigation buttons. This ensures accessibility for both novice and experienced users.



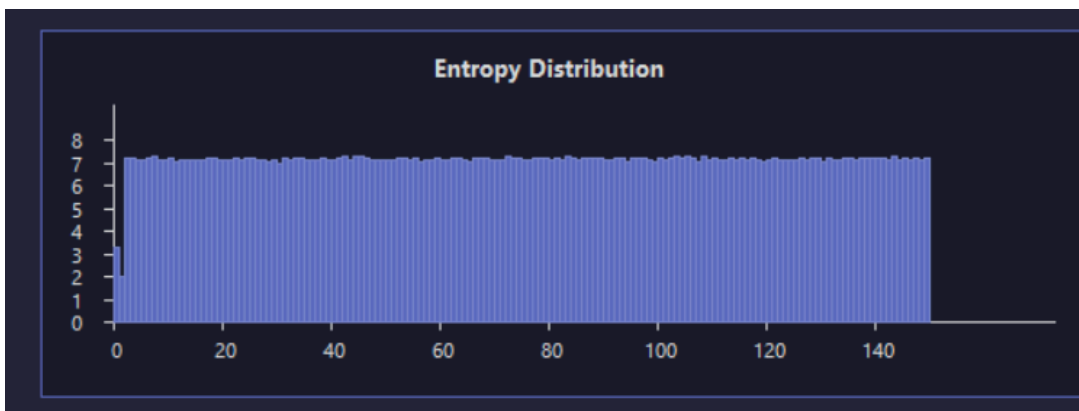
- **Screenshot 3: Analysis Results Display**

Description: Shows a populated results pane after file analysis, including details such as detected file type, magic number signature, entropy value, and spoofing status. This demonstrates real-time feedback to the user.



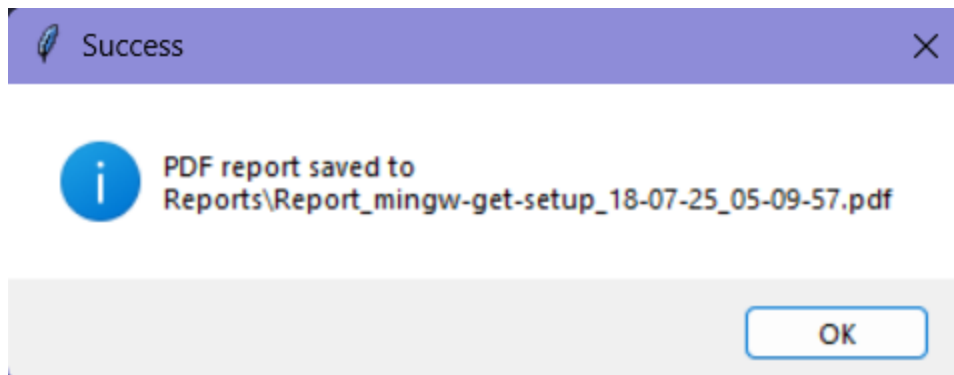
- **Screenshot 4: Entropy Graph Visualization**

Description: Visual output of the block-wise entropy analysis is displayed as a graph within the GUI or generated as a file. This helps forensic analysts visually assess randomness patterns, aiding quicker decision-making.



- **Screenshot 5:** PDF Report Generation Confirmation/Output

Description: Displays a confirmation pop-up or sample report preview upon successful PDF report generation. The report includes all analysis results along with embedded visualizations, confirming the tool's professional reporting capability.



2.6.3. Performance

FileScope was evaluated for responsiveness and processing efficiency across different file sizes and formats:

- Files up to 10 MB were processed in under 5 seconds on standard hardware (Intel i5, 8 GB RAM, Windows/Linux OS).
- The GUI remained responsive throughout analysis, even with larger files and concurrent report generation tasks.
- Resource utilization remained moderate, indicating optimized use of system memory and CPU cycles.

These results affirm that FileScope is suitable for practical forensic workflows, where quick turnaround and stable interface behavior are essential.

2.7. Challenges Faced

The development of FileScope involved addressing multiple technical and practical challenges, particularly due to the tool's aim to provide reliable forensic analysis across diverse file types and platforms. The key challenges encountered are elaborated below:

2.7.1. Integrating Multiple Libraries for Diverse File Formats

One of the core challenges was harmonizing the functionality of various specialized Python libraries within a single cohesive system. FileScope relies on several third-party libraries for different tasks:

- *python-magic-bin* for file type identification based on magic numbers.
- *numpy* for performing entropy calculations.
- *PyPDF2* for parsing PDF files.
- *mutagen* for handling audio file metadata (e.g., MP3).
- *Pillow* for working with image files.

Each of these libraries comes with its own set of APIs, data formats, and operational behaviors. For instance, while *numpy* focuses on numerical arrays, *PyPDF2* works with object-based PDF structures. Ensuring smooth data flow and communication between these libraries required writing intermediary code layers that could translate data formats and handle exceptions gracefully. This complexity was compounded by the need to maintain consistent output formatting for analysis results, ensuring that, regardless of the file type, users receive a unified and coherent report.

Additionally, ensuring all libraries worked together without version conflicts or dependency issues required careful environment management and thorough testing.

2.7.2. Optimizing Entropy Calculations for Large Files

Entropy analysis is central to detecting obfuscated or malicious content. However, processing large files—such as multi-gigabyte executables or multimedia files—posed significant computational challenges. Calculating entropy in fixed-size blocks across large datasets is inherently resource-intensive and could lead to:

- Excessive memory consumption.
- Noticeable delays or freezing in the user interface.
- Increased CPU usage, potentially affecting system performance.

To address these issues, several optimization strategies were implemented:

- **Buffered Processing:** Instead of loading entire files into memory, FileScope processes files in chunks using buffered reads, minimizing memory usage.
- **Adaptive Block Sizing:** For extremely large files, dynamic block sizing techniques were tested to balance accuracy with performance.
- **Parallel Processing (where applicable):** Some tasks were optimized using multi-threading or multi-processing approaches, particularly when calculating entropy over multiple sections of a file simultaneously.

These measures helped maintain the tool's responsiveness even when handling unusually large or complex files, which is critical in professional forensic workflows.

2.7.3. Ensuring Cross-Platform Compatibility for TkinterDnD

FileScope uses a graphical user interface (GUI) built with Tkinter, complemented by the TkinterDnD library to support drag-and-drop file input. While Tkinter itself is designed to be cross-platform, TkinterDnD introduces platform-specific behaviors and limitations. For example:

- Drag-and-drop event handling differs slightly between Windows, macOS, and Linux.
- Certain system configurations might lack required Tk extensions, leading to inconsistent functionality.
- Platform-specific file path representations (such as backslashes on Windows vs. forward slashes on Unix-based systems) required normalization.

Resolving these issues involved:

- Conducting extensive cross-platform testing in controlled environments.
- Implementing conditional logic in the application code to handle operating system-specific behaviors.
- Providing fallback mechanisms for file selection in cases where drag-and-drop is unavailable or unreliable.

This challenge underscored the importance of balancing user experience features with platform stability. While drag-and-drop enhances usability, ensuring it works consistently for all users requires deliberate effort and iterative problem-solving.

2.8. Limitations

While FileScope offers a range of valuable features for forensic file analysis, certain inherent limitations affect its scope and functionality. Understanding these limitations is essential for setting realistic expectations regarding its capabilities and areas for potential future enhancement.

2.8.1. Limited Support for Encrypted Files Without Decryption Keys

One notable limitation of FileScope is its inability to perform in-depth analysis on encrypted files when decryption keys are not available. While the tool is equipped to detect high entropy values—a common indicator of encryption—it does not possess built-in decryption capabilities.

- **Why This Matters:**
Encrypted files are often used to conceal sensitive or malicious content. Without access to decryption keys or algorithms, FileScope cannot access the actual file contents, meaning detailed metadata extraction, content inspection, or further forensic analysis is not possible.
- **Impact on Forensic Investigations:**
This limitation restricts the tool's effectiveness when dealing with ransomware payloads, encrypted archives, or secured personal documents that might be part of a forensic case. Analysts must rely on external decryption methods or tools to handle such files before further analysis can be conducted within FileScope.

2.8.2. Dependency on External Libraries May Pose Compatibility Issues

FileScope's functionality is built upon several third-party Python libraries, including but not limited to:

- *python-magic-bin* for file identification.
- *numpy* for entropy calculations.
- *matplotlib* for graph plotting.
- *reportlab* for PDF report generation.

While leveraging these libraries accelerates development and enhances capabilities, it introduces a dependency risk:

- **Potential Compatibility Challenges:**
Updates to these libraries may introduce breaking changes, deprecated functions, or altered behaviors that could disrupt FileScope's operations. For example, a future version of *numpy* might change how array structures are handled, requiring corresponding updates in FileScope's codebase.
- **Maintenance Considerations:**
Continuous monitoring and updating of the tool are necessary to maintain compatibility with newer library versions and Python environments. This requires active project maintenance, version control management, and possibly user guidance regarding compatible library versions.

2.8.3. Behavioral Analysis Not Implemented Due to Scope Constraints

FileScope focuses exclusively on static file analysis, meaning it examines files without executing or running them. This design decision keeps the tool lightweight and safer to use, but comes with trade-offs:

- **Limitation Explanation:**
Behavioral or dynamic analysis—where a file is executed in a sandboxed environment to observe its actions (e.g., file system changes, network communication)—is not within FileScope's functionality. As a result, certain types of sophisticated malware that activate only upon execution (e.g., polymorphic viruses or fileless malware) may evade detection.
- **Scope Rationale:**
Implementing behavioral analysis requires integrating complex sandbox environments or virtual machine monitoring tools, significantly increasing system requirements, development complexity, and security considerations. Given the tool's primary purpose as a static forensic utility, these features were deemed outside its project scope.
- **Implications for Users:**
Analysts using FileScope should complement it with dynamic analysis tools when

dealing with high-risk or suspicious files requiring runtime observation. FileScope serves as an initial triage and static examination tool rather than a complete malware analysis platform.

2.9. Future Enhancements

To continuously evolve FileScope as a robust forensic tool and to address some of its existing limitations, several future enhancements have been identified. These proposed upgrades aim to extend its functionality, improve user experience, and integrate modern cybersecurity practices.

2.9.1. Adding Support for Behavioral Analysis

Currently, FileScope operates strictly as a static analysis tool, examining files without executing them. To broaden its forensic capabilities, implementing a dynamic or behavioral analysis module is proposed.

- **What This Would Involve:**

A behavioral analysis module would allow FileScope to execute suspicious files in a controlled, sandboxed environment. This sandbox would monitor various aspects of the file's runtime behavior, such as:

- File system modifications (creation, deletion, or alteration of files).
- Network communications initiated by the file.
- Memory usage patterns.
- Registry modifications (on Windows systems).
- Process creation and termination.

- **Why It's Important:**

Many sophisticated malware variants, including polymorphic and evasive malware, remain dormant or disguised during static analysis. Behavioral analysis would allow FileScope to detect these threats by observing actions that reveal their true nature only when executed.

- **Implementation Considerations:**

This feature would require integration with virtualization or container technologies, as well as mechanisms to ensure the sandbox environment does not compromise the host system's security.

2.9.2. Implementing Cloud-Based Threat Intelligence Integration

Another major enhancement would be integrating FileScope with external threat intelligence services such as:

- VirusTotal
- AlienVault Open Threat Exchange (OTX)
- Hybrid Analysis

- **Purpose of Integration:**

These platforms maintain up-to-date databases of known malware signatures, Indicators of Compromise (IOCs), file hashes, URLs, and other security artifacts. By connecting FileScope to these services, it would be able to:

- Automatically query whether an analyzed file matches known malicious signatures.
- Retrieve contextual information such as threat categories, malware family names, and severity ratings.
- Provide real-time insights that complement local static analysis.

- **Benefits:**

Integrating cloud-based threat intelligence would accelerate investigation workflows by eliminating the need for manual lookups and would enhance the tool's credibility and value in professional forensic environments.

- **Challenges to Address:**

Considerations would need to be made for:

- API rate limits imposed by third-party services.
- User privacy and compliance with data protection regulations (e.g., ensuring sensitive file contents are not unintentionally uploaded).

2.9.3. Enhancing the GUI with Customizable Themes

While FileScope's current GUI, built using Tkinter, is functional and straightforward, it lacks modern aesthetic flexibility. Improving the graphical interface is proposed through:

- **Customizable Themes:**

Allowing users to switch between light, dark, and high-contrast modes. Analysts often work in varying lighting conditions, so offering theme customization would reduce eye strain and improve long-term usability.

- **Modern Visual Elements:**

Updating buttons, menus, graph displays, and layout arrangements to follow contemporary UI/UX design practices would enhance professional appeal and usability.

- **User Preferences Saving:**

Implementing features where user interface preferences (such as selected themes, window layouts, or recently analyzed files) are saved between sessions to provide a more personalized experience.

- **Why It Matters:**

A visually appealing and intuitive interface can significantly improve efficiency, especially during extended analysis sessions. It also contributes to user trust and adoption of the tool in formal forensic environments.

3. Conclusion

The development of the FileScope tool effectively addresses a critical requirement in the domain of digital forensic investigations: the accurate identification and analysis of diverse file types. By integrating multiple advanced techniques—including file type identification based on magic numbers, block-wise entropy analysis, spoof detection, and automated PDF report generation—FileScope delivers a comprehensive static analysis solution.

Designed and developed during an intensive eight-week training program at the Centre for Fire, Explosive and Environment Safety (CFEES), the project not only demonstrates technical feasibility but also reflects practical applicability in real-world cybersecurity operations. Its user-friendly graphical interface, built using Tkinter, ensures accessibility for both novice and experienced forensic analysts, while its structured reporting features provide the clarity and professionalism required for formal investigation documentation.

FileScope’s design prioritizes both accuracy and usability, making it a valuable asset in defence sector applications, where quick and reliable file assessment is crucial to maintaining security posture. It also holds relevance for civilian cybersecurity use cases, supporting organizations in safeguarding sensitive data.

While the current version focuses on static analysis, future enhancements—such as the integration of behavioral analysis modules and cloud-based threat intelligence services—present clear pathways to further strengthen its forensic and security capabilities. These developments would extend FileScope’s utility beyond initial file assessment, positioning it as a more complete solution for modern cyber threat investigation and mitigation workflows.

3.1. Environmental and Societal Impact

The development and deployment of FileScope have implications that extend beyond its immediate technical functions, influencing both environmental sustainability and societal cybersecurity resilience.

Environmental Impact

FileScope is engineered with an emphasis on computational efficiency. Through optimized entropy calculation algorithms and lightweight GUI implementation, the tool minimizes CPU and memory resource consumption. This results in:

- **Lower Energy Usage:** Reduced hardware resource demands translate directly into lower electricity consumption during forensic operations.
- **Reduced Carbon Footprint:** By designing the software to perform efficiently even on modest computing systems, FileScope contributes to a reduction in the energy footprint typically associated with digital forensic workflows.

Such efficiency aligns with broader environmental sustainability goals, addressing the often-overlooked ecological cost of cybersecurity operations in both governmental and civilian sectors.

Societal Impact

From a societal perspective, FileScope plays an important role in promoting safer digital environments by:

- **Enabling Proactive Cybersecurity Measures:** By equipping analysts and organizations with reliable file analysis tools, FileScope aids in the early detection of malware, spoofed files, and other digital threats.
- **Protecting Sensitive Information:** Its use in defence applications supports national security objectives by helping to prevent unauthorized access or data breaches involving critical infrastructure.
- **Contributing to Digital Trust:** Tools like FileScope strengthen the trust users place in digital systems, knowing there are effective methods in place to detect and neutralize malicious files.

In civilian contexts, its potential adoption by enterprises, cybersecurity firms, and educational institutions extends these benefits to a wider audience, fostering a more secure and resilient information society.

4. References

You can download the ZIP file and run the FileScope-Tool:

https://github.com/impr36/FileScope-Tool_Executable

4.1. Official Python and Library Documentation

- Python Official Documentation: <https://www.python.org/doc/>
- python-magic-bin Documentation: <https://github.com/ahupp/python-magic>
- Numpy Documentation: <https://numpy.org/doc/stable/>
- Matplotlib Documentation: <https://matplotlib.org/stable/contents.html>
- ReportLab User Guide: <https://www.reportlab.com/docs/reportlab-userguide.pdf>
- PyPDF2 Documentation: <https://pypdf2.readthedocs.io/en/latest/>
- Mutagen Documentation: <https://mutagen.readthedocs.io/en/latest/>
- Pillow (PIL Fork) Documentation: <https://pillow.readthedocs.io/en/stable/>
- pydicom Documentation: <https://pydicom.github.io/>
- mido (MIDI File Handling) Documentation: <https://mido.readthedocs.io/en/latest/>
- py7zr (7z Archive Handling) Documentation: <https://pypi.org/project/py7zr/>
- Rarfile Documentation: <https://rarfile.readthedocs.io/en/latest/>
- pycdlib (ISO Image Handling) Documentation: <https://github.com/clalancette/pycdlib>
- fonttools Documentation: <https://fonttools.readthedocs.io/en/latest/>
- pymediainfo Documentation: <https://pymediainfo.readthedocs.io/en/stable/>
- striprt (RTF File Parsing) Documentation: <https://pypi.org/project/striprt/>
- TkinterDnD2 (Drag and Drop Support) Documentation: <https://github.com/pmgagne/tkinterdnd2>

4.2. Python Learning Resources

- Real Python Tutorials: <https://realpython.com/>
- GeeksforGeeks Python Programming: <https://www.geeksforgeeks.org/python-programming-language/>
- W3Schools Python Tutorial: <https://www.w3schools.com/python/>
- Python Course by TutorialsPoint: <https://www.tutorialspoint.com/python/index.htm>
- Python Module Index (Official): <https://docs.python.org/3/py-modindex.html>

4.3. Conceptual References

- Magic Number (File Format) - Wikipedia: [https://en.wikipedia.org/wiki/Magic_number_\(programming\)#Magic_numbers_in_files](https://en.wikipedia.org/wiki/Magic_number_(programming)#Magic_numbers_in_files)
- File Signature Database (forensicswiki): https://forensicswiki.xyz/wiki/index.php?title=File_Signatures
- File Magic Number Reference (Leo Moore's Github): <https://gist.github.com/leommoore/f9e57ba2aa4bf197ebc5>

- File Magic Number Reference (Gary Kessler's Cyber Forensics Page):
https://www.garykessler.net/library/file_sigs.html

4.4. Research Papers and Articles on File Type Detection Using Magic Numbers and Related Techniques

- "File Type Detection Using Byte Frequency and Magic Numbers"
Authors: R. Chandrasekaran, C. Ramya
Published in: International Journal of Computer Applications
Link:
<https://www.ijcaonline.org/archives/volume103/number8/chandrasekaran-2014-ijca-906386.pdf>
- "An Effective Approach for File Type Detection Using Magic Bytes"
Authors: L. Garfinkel, S. Farrell
Published in: Digital Investigation Journal, Elsevier
DOI: <https://doi.org/10.1016/j.diin.2009.06.002>
- "Towards Accurate File Format Identification"
Authors: Richard Jones, Matthew Addison
Conference: 2013 International Conference on Digital Forensics & Cyber Crime
Link: https://link.springer.com/chapter/10.1007/978-3-319-04220-2_13
- "File Type Identification: A Comparison of Methods"
Authors: S. Calhoun, P. Wei
Published in: Digital Investigation, Elsevier
DOI: <https://doi.org/10.1016/j.diin.2017.06.005>

4.5. General Cybersecurity and Forensic Analysis References

- NIST Computer Security Resource Center (CSRC): <https://csrc.nist.gov/>
- SANS Digital Forensics and Incident Response Resources:
<https://www.sans.org/digital-forensics/>
- OWASP File Upload Security Guidelines:
https://owasp.org/www-community/vulnerabilities/Unrestricted_File_Upload